| | | |
|---|---|---|
| | Cours : Web Avancé et Développement Mobile | **Année universitaire :** 2024/2025 |

# Développement Mobile

**TP4 : Ergonomie d'une application Android**

## Partie 1 – Création d'un Menu & Toasts

### Introduction :

Les menus sont des éléments importants dans les applications Android, car ils offrent aux utilisateurs un moyen pratique d'accéder aux fonctionnalités et aux options de l'application. Dans ce tutoriel, nous allons apprendre à créer un menu dans une application Android en utilisant le fichier de menu XML.

- Rappel sur les menus :

Les menus dans une application Android sont généralement définis dans des fichiers XML situés dans le répertoire res/menu/ du projet. Ces fichiers XML décrivent les différents éléments du menu, tels que les options de menu et les sous-menus, ainsi que leurs actions associées.

Étapes pour créer un menu dans une application Android :

1. Créer un fichier de menu XML :

   - Tout d'abord, créez un nouveau fichier XML dans le répertoire res/menu/ de votre projet. Par exemple, vous pouvez nommer ce fichier main_menu.xml.
   - Dans ce fichier XML, définissez les éléments de menu que vous souhaitez inclure dans votre application. Chaque élément de menu est représenté par une balise <item> avec des attributs tels que android:id, android:title, et app:showAsAction.

2. Définir les options de menu :

- Lors de la définition des éléments de menu, vous pouvez spécifier différentes options pour chaque élément. Par exemple, vous pouvez utiliser l'attribut

app:showAsAction pour spécifier comment l'élément de menu doit être affiché dans la barre d'outils de l'application. Voici les différentes valeurs que peut prendre cet attribut et leur utilisation :

- always : L'élément de menu est toujours affiché comme une action dans la barre d'outils, même s'il n'y a pas assez d'espace disponible. Il est généralement utilisé pour les actions importantes ou fréquemment utilisées.
- ifRoom : L'élément de menu est affiché comme une action dans la barre d'outils si suffisamment d'espace est disponible. Sinon, il est affiché dans le menu déroulant. C'est la valeur par défaut de app:showAsAction et est couramment utilisée pour les actions moins importantes ou moins fréquemment utilisées.
- never : L'élément de menu est toujours affiché dans le menu déroulant, même s'il y a suffisamment d'espace disponible dans la barre d'outils. Cette option est utilisée lorsque vous voulez toujours masquer l'action dans la barre d'outils.
- withText : L'élément de menu est affiché comme une action dans la barre d'outils avec un texte explicite en plus de l'icône. Cela garantit que le texte est toujours visible à côté de l'icône, ce qui peut être utile pour améliorer la compréhension de l'utilisateur.
- collapseActionView : Utilisé pour les éléments de menu qui peuvent être expansés en une vue de recherche ou une vue personnalisée. Lorsque cette option est spécifiée, l'élément de menu est affiché sous forme de bouton avec une flèche pour indiquer qu'il peut être étendu.
- ifRoom|withText : Cette combinaison des valeurs ifRoom et withText indique que l'élément de menu doit être affiché comme une action dans la barre d'outils si suffisamment d'espace est disponible, avec un texte explicite en plus de l'icône.

3. Inflater le menu dans l'activité :
   - Dans votre activité, vous devez implémenter la méthode onCreateOptionsMenu() pour inflater le menu XML dans la barre d'outils de l'application.
   - Utilisez la méthode getMenuInflater().inflate(R.menu.main_menu, menu) pour inflater le fichier XML de menu dans l'activité.

4. Gérer les actions du menu :
   - Pour gérer les actions lorsque l'utilisateur sélectionne un élément de menu, vous devez implémenter la méthode onOptionsItemSelected().
   - Dans cette méthode, utilisez des instructions conditionnelles pour déterminer quelle action exécuter en fonction de l'élément de menu sélectionné par l'utilisateur.

- Rappel sut les Toasts :

  Les Toasts sont des messages d'alerte temporaires qui apparaissent à l'écran pour informer rapidement l'utilisateur d'une action ou d'un événement spécifique. Ils sont souvent utilisés pour afficher des notifications, des messages de succès ou

d'erreur, ou d'autres informations importantes à l'utilisateur. Voici un rappel sur les Toasts et comment les utiliser dans une application Android :

- Pour créer un Toast, vous devez d'abord obtenir une instance de la classe Toast en appelant la méthode statique makeText() de la classe Toast. Cette méthode prend plusieurs paramètres, y compris le contexte de l'application, le texte à afficher et la durée d'affichage du Toast.

```
Toast.makeText(context, "Message à afficher", Toast.LENGTH_SHORT).show();
```

- Une fois que vous avez créé un Toast, vous pouvez l'afficher en appelant la méthode show(). Cela affichera le Toast à l'écran pour une durée spécifiée.

```
toast.show();
```

- La durée d'affichage d'un Toast peut être spécifiée en utilisant l'une des constantes LENGTH_SHORT ou LENGTH_LONG de la classe Toast. LENGTH_SHORT affiche le Toast pendant une courte période de temps (généralement quelques secondes 1s-3s), tandis que LENGTH_LONG affiche le Toast pendant une période plus longue(3s-5s).

- Vous pouvez personnaliser l'apparence d'un Toast en définissant une vue personnalisée à afficher à la place du texte par défaut. Pour ce faire, vous devez d'abord créer une vue personnalisée à l'aide de XML ou de code, puis appeler la méthode setView() sur l'instance de Toast pour définir cette vue personnalisée.

```
toast.setView(view);
```

## Exercie 1:

1. Créer un nouveau projet Android ou on va ajouter un Menu dedans.
2. Pour Créer le Menu, créez le fichier main_menu.xml sous le répertoire menu qui va être un répertoire des différents menu que vous allez créer.
3. Ajouter le code suivant pour mettre le contenu de votre menu (figure 1).

```xml
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto">

    <item android:id="@+id/menu_other"
        android:orderInCategory="100"
        app:showAsAction="never"
        android:title="@string/other_item" />

    <item android:id="@+id/menu_settings"
        android:orderInCategory="100"
        app:showAsAction="never"
        android:title="@string/settings_item" />
</menu>
```

*Figure 1 - Contenu du fichier menu*

4. Pour insérer le menu dans l'activité ajouter la méthode onCreateOptionsMenu(Menu menu).

```java
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    getMenuInflater().inflate(R.menu.main_menu, menu);
    return true;
}
```

5. Lancer l'application pour voir le Menu sur l'application.
   Si vous n'arrivez pas à voir le menu sur l'écran il faut ajouter un Toolbar pour insérer le menu dedans. Voici les étapes à suivre :

   - Ajouter un Toolbar avec le code suivant dans activity_main.xml :

```xml
<androidx.appcompat.widget.Toolbar
    android:id="@+id/toolbar"
    android:layout_width="match_parent"
    android:layout_height="?attr/actionBarSize"
    android:background="?attr/colorPrimary"
    android:elevation="4dp"
    tools:ignore="MissingConstraints" />
```

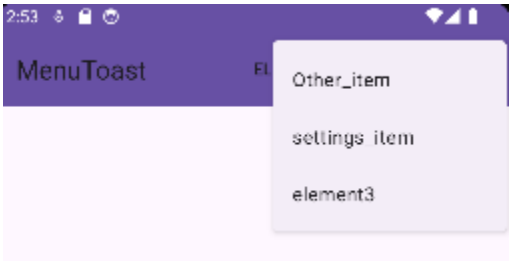
   - N'oubliez pas d'inclure ce fichier Toolbar dans votre activité en appelant setSupportActionBar(toolbar) dans la méthode onCreate() de votre activité

```
Toolbar toolbar = findViewById(R.id.toolbar);
setSupportActionBar(toolbar);
```

6. Pour Gérer les actions sur le menu, on va utiliser simplement des Toast pour afficher du texte sur l'écran. Il faut passer par la méthode onOptionsItemSelected() et récupérer l'id de l'élément qui a été cliquer. Cette méthode dois être ajouté dans le code de votre activité.

```
@Override
public boolean onOptionsItemSelected(@NonNull MenuItem item) {


    int menuItemId = item.getItemId(); // Obtenez l'ID de l'élément de menu
    if (menuItemId == R.id.menu_other) {
        Toast.makeText( context: this,  text: "other Menu Selected", Toast.LENGTH_SHORT) .show();

    } else if (menuItemId == R.id.menu_settings) {
        Toast.makeText( context: this,  text: "Settings Menu Selected", Toast.LENGTH_SHORT) .show();

    } else {

        return super.onOptionsItemSelected(item);
    }
    return super.onOptionsItemSelected(item);
}
```

7. Lancer l'application et tester les clics sur le menu.
8. Ajouter 4 autres éléments de votre choix dans le menu, modifier la valeur de showAsAction afin de faire afficher au moins deux éléments visible sur la barre et les autres en cliquant sur les trois points(Les nouveaux éléments ne doivent pas avoir l'action Never).




9. Ajouter le code correspondant pour gérer le clic sur ces nouveaux éléments.

# Partie 2 – Fragments & Dialogues

## Introduction :

Les fragments sont des éléments essentiels dans le développement d'applications Android, car ils permettent de créer des interfaces utilisateur modulaires et réutilisables. Voici un rappel sur les fragments et comment les utiliser dans une application Android :

- **Qu'est-ce qu'un fragment ?**

Un fragment représente un comportement ou une partie de l'interface utilisateur dans une activité. Il peut être considéré comme une sous-activité modulaire qui a son propre cycle de vie, son propre comportement et sa propre interface utilisateur.

**Utilisation des fragments :**

- Création d'un fragment :

Pour créer un fragment, vous devez créer une classe qui étend la classe Fragment ou l'une de ses sous-classes (DialogFragment, ListFragment, etc.). Vous pouvez définir le comportement du fragment dans les méthodes telles que onCreateView(), onActivityCreated(), onPause(), etc.

- Ajout d'un fragment à une activité :

Vous pouvez ajouter un fragment à une activité en utilisant une instance de FragmentManager. Utilisez les méthodes beginTransaction(), add(), replace() et commit() pour ajouter, remplacer ou supprimer des fragments dans une activité.

- Communication entre fragments et activités :

Les fragments peuvent communiquer avec leur activité parente en utilisant une interface définie par l'activité. L'activité peut ensuite communiquer avec d'autres fragments ou effectuer d'autres actions en fonction de l'interaction avec le fragment.

- Gestion du cycle de vie :

Les fragments ont leur propre cycle de vie, similaire à celui des activités. Assurez-vous de gérer correctement le cycle de vie des fragments en implémentant les méthodes appropriées, telles que onCreateView(), onPause(), onResume(), etc.

- Création de mises en page de fragments :

Vous pouvez définir l'interface utilisateur d'un fragment en créant une mise en page XML dans le dossier res/layout/ de votre projet, puis en l'inflant dans la méthode onCreateView() du fragment.

- **Qu'est-ce qu'un dialogues ?**

Les dialogues sont des composants d'interface utilisateur utilisés pour afficher des messages, des alertes ou des interactions avec l'utilisateur. Voici un rappel sur les dialogues et comment les utiliser dans une application Android :

- Dialogues d'alerte (AlertDialog) :

Les dialogues d'alerte sont utilisés pour afficher un message d'alerte à l'utilisateur, généralement avec des boutons d'action tels que "OK" ou "Annuler". Ils peuvent être utilisés pour demander à l'utilisateur de confirmer une action ou de prendre une décision.

- Dialogues de sélection (Dialog) :

Les dialogues de sélection sont utilisés pour afficher une liste d'options à l'utilisateur, à partir de laquelle il peut sélectionner une ou plusieurs options. Ils peuvent être utilisés pour obtenir une entrée de l'utilisateur ou pour lui permettre de choisir parmi plusieurs options.

- Dialogues personnalisés (CustomDialog) :

Les dialogues personnalisés permettent de créer des interfaces utilisateur personnalisées avec des composants tels que des boutons, des champs de texte, des images, etc. Ils offrent une grande flexibilité pour créer des expériences utilisateur uniques.

**Utilisation des dialogues :**

- Création d'un dialogue :

Pour créer un dialogue, vous devez créer une instance de la classe AlertDialog.Builder (ou une classe dérivée) et définir ses propriétés telles que le titre, le message et les boutons d'action.

- Affichage du dialogue :

Une fois que vous avez configuré le dialogue avec toutes les propriétés nécessaires, vous pouvez l'afficher en appelant la méthode show() sur l'instance du constructeur de dialogue.

- Gestion des interactions avec le dialogue :

Vous pouvez écouter les interactions de l'utilisateur avec le dialogue en définissant des écouteurs d'événements tels que DialogInterface.OnClickListener pour les boutons d'action, DialogInterface.OnCancelListener pour l'annulation du dialogue, etc.

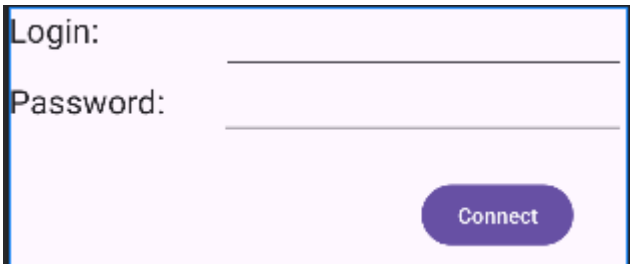
- Personnalisation du dialogue :

Vous pouvez personnaliser l'apparence et le comportement du dialogue en ajoutant des boutons d'action, des options de liste, des cases à cocher, etc. Utilisez les méthodes de la classe AlertDialog.Builder pour ajouter ces éléments au dialogue.

```java
AlertDialog.Builder builder = new AlertDialog.Builder(context);
builder.setTitle("Titre du dialogue");
builder.setMessage("Message du dialogue");
builder.setPositiveButton("OK", new DialogInterface.OnClickListener() {
    public void onClick(DialogInterface dialog, int which) {
        // Action à effectuer lorsque l'utilisateur appuie sur le bouton "OK"
    }
});
builder.setNegativeButton("Annuler", new DialogInterface.OnClickListener() {
    public void onClick(DialogInterface dialog, int which) {
        // Action à effectuer lorsque l'utilisateur appuie sur le bouton "Annuler"
    }
});
builder.show();
```

## Exercie 2:

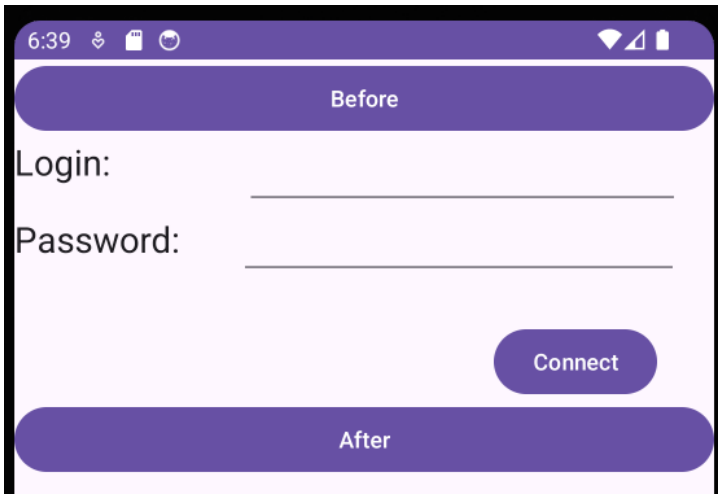
1. Créer un nouveau projet Android ou on va ajouter un nouveau Fragment (Vider le code qui se trouve à l'intérieur de la méthode BlankFragment1, car on va mettre notre propre code).
2. La création du fragment va aussi créer une interface layout qui s'appelle fragment_blank.xml. Dans cette page mettez le code pour créer cette interface :




3. Dans la class BlankFragement, ajouter le code suivant pour l'associer avec sont interface.

```java
@Nullable
@Override
public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container, @Nullable Bundle savedInstanceState) {

    View rootView = inflater.inflate(R.layout.fragment_blank, container, attachToRoot: true);

}
```

4. Dans layout activity_main. Ajouter un bouton avec le texte Before, puis ajouter un fragment (sélectionner celui que vous avez créer), puis ajouter un deuxième bouton. Le résultat doit être comme suit :
   Pour que le contenu du fragment soit visible dans le visualiser ajouter cette ligne dans le code xml du fragment. tools:layout="@layout/fragment_blank"




5. Lancer l'application pour voir si le layout fait appel au fragment.
6. Adapter le code du fragment pour ajouter un évènement sur le click et écrire dans le fichier log.

```
public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container, @Nullable Bundle savedInstanceState) {

    View rootView = inflater.inflate(R.layout.fragment_blank, container, attachToRoot: true);

    txtLogin = (EditText) rootView.findViewById(R.id.txtLogin);
    txtPassword = (EditText) rootView.findViewById(R.id.txtPassword);
    btnConnect = (Button) rootView.findViewById(R.id.btnConnect);

    btnConnect.setOnClickListener(btnConnectListener);
    return rootView;

}

1 usage
private View.OnClickListener btnConnectListener = new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Log.i( tag: "DEBUG", msg: txtLogin.getText() + " / "+ txtPassword.getText());
    }
};
```

7. Modifier le code sur le click pour afficher une boite de dialogue qui affiche les informations que l'utilisateur a saisie avec deux boutons.